

Reviewer Pack — Minimal Rank of Quasi-morphism Invariants in Algebraic Topol...

Entry ea4e3d230b48 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 18:36:01 UTC

Minimal Rank of Quasi-morphism Invariants in Algebraic Topology vs Circuit Lower Bounds for XOR

Entry ID: ea4e3d230b48 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 18:36:01 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Quasi-morphisms) **Field B** (complexity object): Complexity Theory: XOR Circuit Complexity

Statement:

[‘For any n -vertex graph G , let $\rho(G)$ be the smallest number such that there exists a quasi-morphism f from G to the circle that induces an isomorphism on each connected component. Any circuit C computing XOR for all inputs of length n has size at most $(1/\rho(G))^2$.’, ‘Any graph G with $\rho(G) \geq 10$ has an XOR circuit C of size at least 100.’, ‘A quasi-morphism invariant that correctly classifies the minimal rank of any graph G implies a lower bound on the size of any XOR circuit computing XOR for all inputs of length n .’]

Rationale (proposer’s reasoning):

[‘Quasi-morphisms provide a bridge between algebraic topology and graphs, capturing the connectivity of a space. This conjecture links the structure captured by quasi-morphisms to the complexity of circuits that compute XOR, a fundamental operation in complexity theory.’, ‘Algebraic topology has not been extensively used to analyze circuit complexity, particularly for XOR functions. If such a link exists, it could reveal new insights into the nature of XOR-computability.’, ‘This conjecture is testable because the minimal rank of a graph can be computed using software packages like SnapPy, and the size of XOR circuits can be determined by brute-force search or established complexity-theoretic algorithms.’]

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7867cae497286ec2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

A quasi-morphism invariant is supported if, for at least 80% of the graphs G ($n \leq 40$), the correlation between the minimal rank $\rho(G)$ and the XOR circuit size C meets: $|C -$

$(1/\rho(G))^2 \leq 3$. It is falsified if any seed produces a graph with $\rho(G) \geq 10$ and an XOR circuit size $C \geq 100$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'quasi-morphism invariants algebraic topology' AND 'XOR circuit complexity' - 'minimal rank quasi-morphisms' near 'circuit lower bounds XOR' - 'isomorphism induced by quasi-morphism' AND 'size of XOR circuits'

Top relevant hits considered: - [http://arxiv.org/abs/math/0502195v2] Hopf algebra structure on topological Hochschild homology - [http://arxiv.org/abs/math/0302340v2] Pure homology of algebraic varieties - [http://arxiv.org/abs/1612.03133v3] Topological Complexity of the Klein bottle - [http://arxiv.org/abs/2504.19966v3] Quantum circuit lower bounds in the magic hierarchy - [http://arxiv.org/abs/1806.02734v3] Spectral lower bounds for the orthogonal and projective ranks of a graph - [http://arxiv.org/abs/1709.02311v2] A Tight Lower Bound for Counting Hamiltonian Cycles via Matrix Rank - [http://arxiv.org/abs/1904.06128v1] Situationally-Induced Impairments and Disabilities Research - [http://arxiv.org/abs/math/0106093v2] On the Complexity of Polytope Isomorphism Problems

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    rows = len(A)
    cols = len(A[0])
    for i in range(rows):
        # Find pivot
        max_row = i
        for r in range(i+1, rows):
            if abs(A[r][i]) > abs(A[max_row][i]):
                max_row = r
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below
        factor = Fraction(1, A[i][i])
```

```

    for j in range(i+1, cols):
        A[i][j] *= factor
    A[i][i] = 1

    # Eliminate above
    for r in range(rows):
        if r != i:
            factor = Fraction(A[r][i], A[i][i])
            for j in range(i, cols):
                A[r][j] -= factor * A[i][j]
            A[r][i] = 0

    # Back substitution
    for i in range(rows-1, -1, -1):
        for j in range(i+1, rows):
            A[i][cols-1] -= A[j][cols-1] * A[i][j]
        A[i][cols-1] /= A[i][i]

    return [row[cols-1] for row in A]

def compute_rho(G):
    n = len(G)
    A = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            if G[i][j]:
                A[i][j] = Fraction(1, math.sqrt(2))
                A[j][i] = Fraction(1, math.sqrt(2))

    rank = len(gaussian_elimination(A))
    return rank

def brute_force_xor_circuit(n):
    # This is a very naive approach and will not work for large n
    if n == 1:
        return 1
    return 2 * brute_force_xor_circuit(n-1)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    for i in range(n):
        G[i][i] = 0

    rho_G = compute_rho(G)
    C = brute_force_xor_circuit(n)

    metric_name = "XOR Circuit Size"
    metric_value = C
    instances_tested = 1
    conjecture_holds = abs(C - (1/rho_G)**2) <= 3
    counterexample = "" if conjecture_holds else f"rho(G)={rho_G}, C={C}"

    return {
        "metric_name": metric_name,

```

```

        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2*i + 3 for i in range(5, 8)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(r["counterexample"] != "" for r in results):
        first_failing_seed = next(r["seed"] for r in results if r["counterexample"] != "")
        counterexample_desc = next(r["counterexample"] for r in results if r["counterexample"] != "")
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample_desc}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE no_counterexamples_found")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f7cd606.py", line 98, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f7cd606.py", line 76, in run_trial
    rho_G = compute_rho(G)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4f7cd606.py", line 57, in compute_rho
    A[i][j] = Fraction(1, math.sqrt(2))
                ~~~~~
  File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be ")
TypeError: both arguments should be Rational instances

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying if the conjecture’s support conditions are met. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12566
2	preregistration	ollama_remote	glm4:latest	0	0	5866
3	novelty	ollama_remote	glm4:latest	0	0	4679
4	novelty	ollama_remote	glm4:latest	0	0	8261
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16147
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13144
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14254
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11337
9	judge	ollama_remote	glm4:latest	0	0	11115

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 97369 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/ea4e3d230b48.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/ea4e3d230b48.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/ea4e3d230b48.tar.gz (if generated)